

Use Case 2: Refinery Predictive Maintenance Agent

Segment: Downstream | Claude Code + Bedrock AgentCore Hackathon · Energy Symposium

The Problem

A crude distillation unit (CDU) runs 12 centrifugal pumps continuously, 24/7/365. Each pump has bearing temperature, vibration, pressure, flow, and motor sensors reporting every 5 minutes. When a pump fails unexpectedly, the downstream unit trips, operators scramble, and the refinery loses \$50K–\$200K per hour of unplanned downtime.

The data exists. The sensors are there. But no one has time to watch 144 sensor streams simultaneously, and the CMMS work order system only captures what already broke — not what's about to break. Maintenance teams run on time-based schedules (replace every 12 months regardless of condition) or react after failure. Both approaches are wasteful and dangerous.

Unplanned downtime accounts for up to 5% of annual production losses in refining (IEA). AI-driven predictive maintenance reduces unplanned downtime by 30–50% and maintenance costs by 25% — results reported by BP, Shell, and Saudi Aramco. The key shift is from time-based schedules to condition-based, AI-driven intervention. The challenge isn't just detecting an anomaly — it's reasoning about it in context: *Is this a real failure precursor or a normal operating transient? What does the maintenance history say? What does the manual recommend? How urgent is this?*

Your Task

Solve the problem above using Agentic AI — built with Claude Code and deployed on Amazon Bedrock AgentCore.

Use the dataset and reference documents provided in this package:

- **Sensor history** — [sensor_timeseries.csv](#) (625,536 rows, 12 CDU pumps, 6 months at 5-minute intervals)
- **Failure ground truth** — [failure_event_log.csv](#) (8 labeled failures), [failure_mode_reference.csv](#)
- **Maintenance record** — [pump_maintenance_history.json](#) (59 records) and [maintenance_work_orders.csv](#) (120 records) — two independent logs, see the warning below

- **Operating context** — `pump_metadata.csv`, `vibration_baselines.csv`, `alarm_history.csv`, `operating_schedule.csv`, `spare_parts_inventory.csv`, `cost_tracking.csv`
- **OEM manuals** (`data/reference_docs/`, `.md` and `.pdf`) — Flowserve PVXM-3, KSB Etanorm 100-080, Sulzer CPT-50: vibration limits, FMEA tables, troubleshooting trees

What you decide

Everything else. What the application actually does, which question it answers and who it answers it for, how many agents and how they divide the work, which orchestration pattern, what each tool does, which AgentCore modules you use, and what the interface looks like — all of that is your team's call.

There is deliberately no worked solution and no capability checklist in this document. Deciding what is worth building from the problem and the data is the hackathon. Two teams solving this well should end up with two visibly different applications.

What “grounded” means here

Every number and claim your application states should trace back to a specific file, row, or passage in the data above — and it should be able to say which one. An answer that sounds expert but cites nothing is worth less than a narrower answer that shows its evidence. If the data doesn't support a conclusion, the right behavior is to say so, not to fill the gap.

Judges score the working demo, the depth of the application's reasoning, and whether its claims trace back to this data.

Dataset Provided

All files live under `data/`.

Two things to check before you write any code. 1. `sensor_timeseries.csv` is **625,536 rows, ~60MB**. Don't paste the whole file into a Claude Code turn or read it into memory in one shot — query it, chunk it, or compute aggregates in a script first. 2. `pump_maintenance_history.json` and `maintenance_work_orders.csv` are **two independent work-order logs, not the same list twice**. The JSON uses `W0-00001`-style IDs (59 records); the CSV uses `W0-0001`-style IDs (120 records) — they're different ID spaces, not a padding difference, and there is zero overlap. Don't join on `work_order_id`. If you need to connect them, match on `pump_id` and nearby dates instead.

1. Sensor Time-Series CSV — `sensor_timeseries.csv`

6 months of 5-minute readings for 12 centrifugal pumps — 625,536 rows, 2025-09-01 00:00 → 2026-02-28 23:55. 12 sensor channels per pump.

Columns:

Column	Description
timestamp	ISO 8601 datetime
pump_id	P-01 through P-12
bearing_temp_c	Bearing housing temperature (°C) — primary failure indicator
vibration_x_mms	Radial vibration X-axis (mm/s RMS, ISO 10816)
vibration_y_mms	Radial vibration Y-axis (mm/s RMS, ISO 10816)
vibration_axial_mms	Axial vibration (mm/s RMS) — detects misalignment
suction_pressure_psi	Inlet pressure — detects cavitation
discharge_pressure_psi	Outlet pressure — detects blockage or wear
differential_pressure_psi	Discharge minus suction — pump head indicator
flow_rate_m3hr	Volumetric flow rate — detects impeller wear
motor_current_amps	Motor electrical current — detects mechanical overload
motor_speed_rpm	Shaft speed — detects slip or drive issues
seal_leak_detected	Boolean — seal leak indicator from proximity sensor
label	normal / pre_failure / failure

2. Failure Event Log CSV — `failure_event_log.csv` — 8 labeled failure events across 6 pumps: `event_id`, `pump_id`, `failure_start_ts`, `failure_mode` (bearing_wear / seal_failure / cavitation / impeller_damage / misalignment), `failure_ts`, `downtime_hrs`, `repair_cost_usd`. The `label` column in the time-series marks the `pre_failure` runway leading into each one.

3. Pump Maintenance History JSON — `pump_maintenance_history.json` — a JSON array of 59 completed service records: `pump_id`, `work_order_id`, `date`, `work_type` (inspection / lubrication / bearing_replacement / seal_replacement / impeller_replacement), `technician_notes` (free text), `parts_replaced`, `lubricant_type`, `hours_since_last_service`. History goes back to 2021. `work_order_id` here does not match item 4's — see the warning above.

4. Maintenance Work Orders CSV — `maintenance_work_orders.csv` — 120 CMMS work orders, 9 columns. The structured work order trail — status, dates, assignment, cost. A separate ID space from item 3's `work_order_id` — see the warning above.

- 5. Pump Metadata CSV** — `pump_metadata.csv` — 12 pumps. `pump_id`, `model`, `manufacturer`, `install_date`, `service_fluid`, `design_flow_m3hr`, `design_head_m`, `rated_power_kw`, `criticality_rating` (1–3), `last_maintenance_date`, `bearing_type`, `seal_type`, `operating_envelope_json`. The `model` field tells you which OEM manual applies to a given pump.
- 6. Failure Mode Reference CSV** — `failure_mode_reference.csv` — 5 rows, one per failure mode: `failure_mode`, `primary_sensor_signature`, `secondary_indicators`, `typical_lead_time_hrs`, `recommended_action`, `iso_10816_zone_at_detection`. The lookup that maps a sensor pattern to a diagnosis.
- 7. Alarm History CSV** — `alarm_history.csv` — 200 historical alarm events across the 12 pumps, 8 columns. Most are nuisance alarms.
- 8. Vibration Baselines CSV** — `vibration_baselines.csv` — 72 rows (per pump, per axis). The established per-pump normal, per axis. Pump P-03's normal is not pump P-09's normal.
- 9. Operating Schedule CSV** — `operating_schedule.csv` — 2,160 rows (per pump, per day). Duty state and load over the dataset window.
- 10. Spare Parts Inventory CSV** — `spare_parts_inventory.csv` — 12 parts. Stock levels and lead times per part.
- 11. Cost Tracking CSV** — `cost_tracking.csv` — 288 rows (per pump, per month). Maintenance spend history.
- 12. Equipment Manuals** — `reference_docs/` — 3 OEM manuals (Flowserve PVXM-3, KSB Etanorm 100-080, Sulzer CPT-50), each supplied as both `.md` and `.pdf`: failure mode tables, bearing life curves, vibration severity charts (ISO 10816 zones A/B/C/D), seal replacement procedures, lubrication intervals, torque specs. The Markdown versions are easier to chunk and index; the PDFs are there if you want to demo document parsing.
-

Logging in, environment setup, and how to submit: see the hackathon portal.

Claude Code: <https://docs.claude.com/en/docs/claude-code> **Strands Agents SDK:** <https://strandsagents.com> **Amazon Bedrock AgentCore:** <https://docs.aws.amazon.com/bedrock-agentcore/>